

Chapter 10 — Memory

Standing context, and why it is not a hippocampus

The model has no hippocampus. So you have to be its hippocampus.



What that means concretely

- Nothing persists between sessions. Chapter 1 said so; Chapter 8 named what is absent.
- Every time a tool appears to remember your project, **something outside the model re-inserted that text** into the window.
- That something is a file. You write it, you own it, and it is the deliverable of today.



Standing context

A project instruction file is read into the window at the start of every session in that project.

- Conventions the code cannot state for itself.
- Build and run commands, with the flags that actually work on your machine.
- **Units, sign conventions and coordinate systems.** The things a reviewer catches and a fresh session cannot infer.
- Known pitfalls — the trap you fell into last month, written down so you do not fall in again.



What does not belong in it

- Anything readable from the code itself. Directory listings, function signatures, what a well-named function obviously does.
- Aspirations. A file describing how you wish the project worked will be applied to the project that exists.
- Anything you would not defend to a colleague reading it cold.

The test: **would a competent new member of your group need this told to them?** If not, it is costing you on every turn and buying nothing.



The cost of memory

Standing context is resident. It is re-read on every turn of every session.

$$\text{cost}_{\text{standing}} \propto N_{\text{file}} \times T_{\text{turns}}$$

N_{file} — tokens in the instruction file. T_{turns} — turns in the session. Both dimensionless counts, so the constant of proportionality carries the currency: \$ token⁻¹ turn⁻¹.

- A file twice as long costs twice as much, on every turn, forever — **before caching**. Standing context is the canonical cached prefix, and Chapter 13 puts the discount on it. The relation is the uncached bound; the direction survives, the magnitude does not.
- It also occupies window that Chapter 5's degradation curve is already competing for.
- So a bloated instruction file is an engineering problem, not an untidiness problem. That is the difference between this and a style guide.

Threads meeting. Cost, from Chapter 1 through Chapter 7, meets memory and context here. Chapter 13 puts a price on the same tokens.



Run the checkup, live

- The built-in setup check audits configuration — and, in recent versions, memory files and unused components.
- Run it on a real instruction file in front of the room and let it flag the duplication.
- Duplication is the normal failure. Files grow by accretion, and nobody deletes.

TODO(capture) — run it on this repository's own instruction file before the session and keep the output. If it flags nothing, that is worth showing too, and worth saying so.



The hierarchy

- **Personal** — how you like to work. Travels with you across projects.
- **Project** — how this work is done. Committed, shared, reviewed like code.
- **Local** — machine-specific paths and settings. Never committed.
- **Not keys.** Keys go in environment variables, never in a file in the project directory — Chapter 13 explains why, and "never committed" depending on a gitignore entry surviving is exactly the assumption that fails.

The middle one is the one that matters to a research group, because it is the only one that survives the person who wrote it.



Generated drafts are a starting point

- You can ask the tool to draft its own instruction file from the project.
- The draft will be structurally correct and substantively thin. It describes what the code shows and misses what only you know.
- The sign conventions are exactly what it cannot infer, and exactly what the file exists for.

Treat the generated file the way Chapter 14 asks you to treat a generated citation: as a candidate, not a result.



Commands and skills

The same idea, made reusable.

- A custom command is a prompt you have stopped retyping.
- A skill is a procedure with its own instructions, invoked by name, that you can commit alongside the code.
- Both are standing context that loads **only when needed**, which is how you keep the resident cost down while still writing things once.



This is lab infrastructure

- A shared instruction file makes six people's conventions into one group's conventions.
 - Committed skills mean the procedure survives the student who wrote it.
 - A convention written down once and applied automatically beats a convention agreed in a meeting.
-

Revisited in Chapter 18 as governance, where the same file becomes the reproducibility record rather than a convenience.



Exercise — the deliverable of the day

Write and commit a project instruction file for your own work.

1. Start from a generated draft. Read every line of it.
2. Delete everything the code already says.
3. Add the units, the sign conventions, and the coordinate system.
4. Add the one pitfall that cost you a week.
5. Commit it. It is now under review like anything else.

You leave today with an artefact. That is the point of this chapter.



Where this leaves us

- The memory thread closes here: bounded buffer, degradation, the gap Chapter 8 names, management, and now standing context you own.
- **It is not the missing organ.** Consolidation writes experience into durable memory; a file re-inserted into a fresh window is re-prompting, and the weights never move. Chapter 8 breaks that analogy on purpose and this chapter does not quietly rebuild it.
- The replacement is a file. It is auditable, versioned and shared, which the window never was.
- And it has a running cost you can now calculate.

Chapter 11 asks what happens when one session is not enough.

